

Proyecto Final de Sistemas Embebidos: Sistema IoT para Monitoreo y Control de un Cultivo de Hortalizas

Curso: Sistemas Embebidos
Ingeniería Mecatrónica
Universidad Santo Tomás – Seccional Bucaramanga

Resumen—Este documento presenta el enunciado del proyecto final del curso de Sistemas Embebidos. La actividad integra los contenidos desarrollados durante el semestre, desde los fundamentos de sistemas embebidos, el levantamiento de requisitos, la partición hardware/software, los protocolos de comunicación y la implementación de soluciones orientadas al Internet de las Cosas (*Internet of Thing IoT*). El proyecto consiste en el diseño e implementación de un sistema embebido IoT para el monitoreo y control de variables ambientales y de sustrato en un cultivo experimental de hortalizas. La solución debe integrar sensores, actuadores, firmware estructurado, comunicación remota y una plataforma de visualización basada en *Firebase*, *MQTT* o una solución IoT equivalente.

Index Terms—Sistemas embebidos, IoT, MQTT, Firebase, agricultura inteligente, riego por goteo.

I. INTRODUCCIÓN

Los sistemas embebidos constituyen una base tecnológica fundamental para el desarrollo de soluciones mecatrónicas modernas, debido a su capacidad de integrar sensores, actuadores, procesamiento local y comunicación con otros sistemas. En el contexto de la Industria 4.0, estos sistemas adquieren mayor relevancia al conectarse con plataformas digitales mediante tecnologías propias del Internet de las Cosas, permitiendo el monitoreo remoto, la adquisición de datos y la toma de decisiones en tiempo real [1], [4].

El curso de Sistemas Embebidos tiene como propósito orientar al estudiante en el diseño y desarrollo de soluciones a problemas específicos, aplicando conceptos tecnológicos relacionados con el hardware, el firmware, los protocolos de comunicación y el software de monitoreo remoto. En este sentido, el proyecto final propone la implementación de un sistema embebido de IoT aplicado a un cultivo experimental de hortalizas, en el que se deben medir variables ambientales y del sustrato, controlar actuadores y visualizar información de forma remota.

La propuesta se articula con las temáticas abordadas durante el curso: fundamentos de sistemas embebidos, requisitos de diseño, partición hardware/software, protocolos de comunicación serial e inalámbrica, arquitectura IoT, escritura y lectura en *Firebase*, publicación y suscripción en *MQTT*, serialización y deserialización de datos, y buenas prácticas de programación.

II. PROPÓSITO DEL PROYECTO

El proyecto final tiene como propósito evaluar la capacidad del estudiante para integrar conocimientos técnicos y científicos en el diseño de una solución embebida funcional. Para ello, se plantea el desarrollo de un sistema capaz de monitorear variables de interés en un cultivo de hortalizas y controlar dispositivos físicos asociados al riego y al control térmico.

El desarrollo debe partir del análisis del problema, la identificación de requisitos funcionales y no funcionales, la selección justificada de una tarjeta de desarrollo, la definición de la arquitectura hardware/software, la implementación del firmware y la integración con una plataforma IoT.

III. PROBLEMA DE INGENIERÍA

En pequeños cultivos de hortalizas, el riego y el control de las condiciones ambientales suelen realizarse de forma manual, lo que puede generar un uso ineficiente del agua, variabilidad en la humedad del sustrato, estrés hídrico en las plantas y una baja capacidad de seguimiento técnico del proceso.

Ante esta situación, se propone desarrollar un sistema embebido de IoT que permita monitorear variables ambientales y del sustrato, activar automáticamente el riego por goteo, controlar un mecanismo asociado a la regulación térmica y visualizar de forma remota el estado del sistema.

IV. OBJETIVO GENERAL

Diseñar e implementar un sistema embebido IoT para el monitoreo remoto y control automático de variables asociadas al crecimiento de hortalizas, integrando sensores, actuadores, comunicación inalámbrica, plataforma de visualización y buenas prácticas de desarrollo de firmware.

V. OBJETIVOS ESPECÍFICOS

1. Definir los requisitos funcionales y no funcionales del sistema embebido IoT, considerando las variables ambientales y de sustrato necesarias para el monitoreo y control del cultivo de hortalizas.
2. Diseñar la arquitectura hardware/software del sistema, justificando la selección de la tarjeta de desarrollo, sensores, actuadores, mecanismos de comunicación y plataforma IoT.
3. Implementar el prototipo funcional del sistema embebido, integrando la adquisición de datos, la lógica de control

para riego y regulación térmica, y la comunicación remota mediante Firebase, MQTT o una solución IoT equivalente.

4. Validar el funcionamiento del sistema mediante pruebas experimentales, documentación técnica, registro del código en GitHub, análisis de resultados y presentación de evidencias del monitoreo y control remoto.

VI. COMPETENCIAS EVALUADAS

El proyecto permite evaluar las siguientes competencias del curso:

VI-A. Competencia C5

Comprender el lenguaje técnico en una segunda lengua con relación a sistemas mecatrónicos para su implementación en el desarrollo de soluciones técnicas.

Esta competencia se evaluará mediante la consulta de hojas de datos, documentación técnica de sensores, librerías, tarjetas de desarrollo, plataformas IoT y protocolos de comunicación.

VI-B. Competencia C3

Desarrollar capacidades en Ciencia, Tecnología e Innovación empleando estrategias de aprendizaje autónomo, apropiación del conocimiento y desarrollo de productos de base tecnológica.

Esta competencia se evidenciará mediante la formulación de una solución tecnológica aplicada a agricultura inteligente, la documentación del proceso, el análisis crítico de resultados y la propuesta de mejoras futuras.

VI-C. Competencia C6

Desarrollar sistemas mecatrónicos que integren conocimientos técnico-científicos para la solución de problemas ingenieriles en procesos industriales.

Esta competencia se evaluará mediante la integración funcional de sensores, actuadores, microcontrolador, firmware, comunicación IoT y plataforma de visualización.

VII. DESCRIPCIÓN GENERAL DEL SISTEMA

El sistema debe permitir el monitoreo y control de un cultivo experimental de hortalizas. Para ello, se deben medir variables ambientales y del sustrato, activar dispositivos físicos y transmitir información hacia una plataforma IoT.

La arquitectura mínima del sistema debe incluir:

- Sensores para adquisición de variables físicas.
- Tarjeta embebida para procesamiento y control.
- Actuadores para riego y control térmico.
- Comunicación inalámbrica o conexión a red.
- Plataforma remota para visualización de datos.
- Repositorio GitHub para control y registro del desarrollo.

VIII. MATERIALES BASE

En la Tabla I se presentan los materiales base considerados para el desarrollo del proyecto.

Cuadro I
MATERIALES BASE DEL PROYECTO

Elemento	Función dentro del sistema
Semillas de hortalizas	Elemento biológico del sistema experimental.
Cinta de riego por goteo	Distribución controlada del agua hacia el cultivo.
Tarjeta electrónica	Procesamiento, control y comunicación del sistema.
Sensor de temperatura ambiente	Medición térmica del entorno.
Sensor de humedad relativa del aire	Medición de condiciones ambientales.
Sensor electroquímico de humedad del sustrato	Medición de humedad en el suelo o sustrato.
Bomba de irrigación	Activación del riego por goteo.
Bomba o actuador térmico	Regulación de temperatura o microambiente.
Techo corredizo	Elemento físico de control ambiental.
Plataforma IoT	Monitoreo remoto y registro de datos.

IX. TARJETAS PERMITIDAS

El grupo podrá seleccionar una de las siguientes plataformas:

- ESP32.
- STM32 Nucleo de 30 pines.
- Raspberry Pi Pico o Raspberry Pi Pico W.

La selección de la tarjeta debe justificarse considerando:

- Número de entradas analógicas y digitales.
- Capacidad de comunicación.
- Compatibilidad con sensores y actuadores.
- Nivel lógico de operación.
- Disponibilidad de librerías.
- Facilidad de programación desde Visual Studio Code.
- Escalabilidad del sistema.
- Consumo energético.

El uso del ESP32 puede ser conveniente en proyectos IoT debido a su conectividad Wi-Fi y Bluetooth integrada [10]. La Raspberry Pi Pico W también permite aplicaciones conectadas mediante Wi-Fi [11], mientras que las tarjetas STM32 Nucleo permiten trabajar con una arquitectura de microcontrolador ampliamente utilizada en aplicaciones industriales y académicas [12].

X. RESTRICCIONES OBLIGATORIAS

El proyecto debe cumplir con las siguientes restricciones:

1. No se permite el uso del Arduino IDE.
2. No se permite entregar archivos con extensión .ino.
3. Todo el desarrollo debe realizarse desde Visual Studio Code.
4. El código fuente debe estar alojado en un repositorio de GitHub.

5. El repositorio debe evidenciar avances mediante commits organizados.
6. El sistema debe incluir al menos tres variables medidas.
7. El sistema debe incluir al menos dos actuadores controlados.
8. El sistema debe contar con monitoreo remoto mediante Firebase, MQTT o una solución IoT equivalente.
9. El código debe presentar comentarios técnicos, nombres de variables claros y organización modular.
10. Se debe evitar el uso de `delay()` en sistemas secuenciales complejos.

XI. VARIABLES DEL SISTEMA

En la Tabla II se presentan las variables mínimas que debe considerar el sistema.

Cuadro II
VARIABLES MÍNIMAS DEL SISTEMA

Variable	Tipo	Uso
Temperatura ambiente	Entrada	Control térmico.
Humedad relativa del aire	Entrada	Monitoreo ambiental.
Humedad del sustrato	Entrada	Activación del riego.
Estado de bomba de riego	Salida	Control de irrigación.
Estado de actuador térmico	Salida	Regulación ambiental.

XII. PARTICIÓN HARDWARE/SOFTWARE

La partición hardware/software corresponde a la decisión sobre qué funciones serán implementadas mediante componentes físicos y cuáles serán resueltas mediante firmware. Esta etapa es fundamental en el diseño de sistemas embebidos, ya que permite distribuir responsabilidades entre sensores, actuadores, circuitos de potencia, microcontrolador y lógica de control [3], [1].

XII-A. Funciones Asociadas al Hardware

- Lectura física de sensores.
- Activación de bombas o actuadores.
- Acondicionamiento de señales.
- Protección eléctrica de cargas.
- Alimentación del sistema.
- Conexión de relés, transistores, MOSFET o drivers.

XII-B. Funciones Asociadas al Software

- Lectura periódica de variables.
- Filtrado básico o validación de datos.
- Comparación con umbrales.
- Implementación de histéresis.
- Publicación de datos en Firebase o MQTT.
- Recepción de comandos remotos, si aplica.
- Registro de estados del sistema.
- Gestión de errores o alarmas.

XIII. LÓGICA DE CONTROL

El sistema debe implementar al menos dos lazos de decisión: uno asociado al riego y otro asociado al control térmico.

XIII-A. Control de Riego

El control de riego debe activarse cuando la humedad del sustrato esté por debajo de un umbral mínimo. Para evitar conmutaciones rápidas de la bomba, se debe implementar histéresis.

```
Si humedad_sustrato < humedad_minima:
    activar bomba de riego
```

```
Si humedad_sustrato > humedad_minima + histeresis:
    desactivar bomba de riego
```

XIII-B. Control Térmico

El control térmico debe activarse cuando la temperatura ambiente supere un valor máximo permitido. El actuador puede ser una bomba, ventilador, techo corredizo u otro mecanismo definido por el grupo.

```
Si temperatura > temperatura_maxima:
    activar actuador termico
```

```
Si temperatura < temperatura_maxima - histeresis:
    desactivar actuador termico
```

XIV. COMPONENTE IOT

El componente IoT es obligatorio. El sistema debe permitir el monitoreo remoto de las variables medidas y de los estados de los actuadores.

XIV-A. Opción A: Firebase

Firebase puede utilizarse para escritura y lectura de datos en tiempo real. El sistema deberá enviar al menos las siguientes variables:

- Temperatura ambiente.
- Humedad relativa del aire.
- Humedad del sustrato.
- Estado de bomba de riego.
- Estado del actuador térmico.

XIV-B. Opción B: MQTT

MQTT es un protocolo ligero basado en el modelo publicador/suscriptor, ampliamente utilizado en aplicaciones IoT e IIoT por su eficiencia en el envío de datos de telemetría [7], [8].

Ejemplos de tópicos MQTT:

```
cultivo/temperatura
cultivo/humedad_aire
cultivo/humedad_sustrato
cultivo/bomba_riego/estado
cultivo/control_termico/estado
cultivo/comando/riego
```

El sistema debe evidenciar la serialización y deserialización de datos. Se recomienda el uso de JSON:

```
{
  "temperatura": 28.5,
  "humedad_aire": 71,
```

```

"humedad_sustrato": 43,
"bomba_riego": "ON",
"control_termico": "OFF"
}

```

XV. GESTIÓN DEL PROYECTO

El desarrollo debe gestionarse mediante una metodología ágil básica, preferiblemente Scrum o Kanban. Las metodologías ágiles permiten organizar el trabajo en ciclos cortos, con entregas parciales, revisión continua y adaptación frente a problemas técnicos [16].

Cada grupo debe presentar:

- Product backlog.
- Historias de usuario.
- Tareas semanales.
- Responsables.
- Evidencias de avance.
- Problemas encontrados.
- Ajustes realizados durante el desarrollo.

Ejemplo de historia de usuario:

Como usuario del sistema de cultivo, quiero visualizar la humedad del sustrato desde una plataforma remota, para verificar si el cultivo requiere riego o si el sistema automático está funcionando correctamente.

XVI. BUENAS PRÁCTICAS DE PROGRAMACIÓN

El firmware debe cumplir con criterios mínimos de calidad:

- Uso de nombres descriptivos.
- Separación de funciones.
- Evitar código repetido.
- Evitar el uso excesivo de `delay()`.
- Uso de temporización no bloqueante.
- Comentarios técnicos donde sea necesario.
- Definición clara de pines.
- Definición de constantes para umbrales.
- Validación de lecturas de sensores.
- Manejo de estados de actuadores.
- Publicación periódica de datos IoT.

Ejemplo de estructura mínima de funciones:

```

void readSensors();
void controlIrrigation();
void controlTemperature();
void publishIoTData();
void updateActuators();

```

XVII. ENTREGABLES

XVII-A. *Entregable 1: Documento de Requisitos y Diseño*

Debe contener:

- Descripción del problema.
- Objetivo general y objetivos específicos.
- Requisitos funcionales.
- Requisitos no funcionales.
- Diagrama general del sistema.
- Selección justificada de la tarjeta.

- Listado de sensores y actuadores.
- Partición hardware/software.
- Diagrama de flujo o máquina de estados.

XVII-B. *Entregable 2: Repositorio GitHub*

Debe incluir:

- Código fuente.
- Archivo `README.md`.
- Diagrama o imagen del sistema.
- Instrucciones de instalación.
- Instrucciones de ejecución.
- Descripción de librerías utilizadas.
- Capturas de funcionamiento.
- Historial de commits.

No se aceptan entregas con un único commit final.

XVII-C. *Entregable 3: Prototipo Funcional*

Debe demostrar:

- Lectura de sensores.
- Activación de actuadores.
- Control de riego.
- Control térmico.
- Envío de datos a plataforma IoT.
- Visualización remota de variables.

XVII-D. *Entregable 4: Bitácora Técnica*

Debe registrar:

- Avances semanales.
- Decisiones de diseño.
- Problemas encontrados.
- Evidencias fotográficas.
- Capturas de la plataforma IoT.
- Pruebas de sensores.
- Pruebas de actuadores.
- Lecciones aprendidas.

XVII-E. *Entregable 5: Presentación Final*

Debe incluir:

- Problema abordado.
- Arquitectura del sistema.
- Hardware utilizado.
- Firmware desarrollado.
- Plataforma IoT implementada.
- Resultados obtenidos.
- Limitaciones.
- Trabajo futuro.
- Demostración del prototipo.

XVIII. CRONOGRAMA DE ENTREGAS PARCIALES

El proyecto final se desarrollará mediante entregas parciales, con el propósito de evidenciar el avance progresivo del diseño, implementación, documentación y validación del sistema embebido IoT. Cada entrega debe estar registrada en la bitácora técnica y respaldada en el repositorio GitHub del grupo.

Todas las evidencias deben incluir fecha, descripción del avance, responsables, fotografías, capturas de pantalla, diagramas, archivos fuente o enlaces correspondientes. No se aceptarán evidencias presentadas únicamente el día de la entrega final sin trazabilidad previa en la bitácora y en GitHub.

Cuadro III
CRONOGRAMA DE ENTREGAS PARCIALES DEL PROYECTO FINAL

Entrega	Fecha	Componentes mínimos a presentar
Primera entrega	Jueves 7 de mayo de 2026	Levantamiento de requisitos, análisis del problema, arquitectura hardware/software y diseño CAD del prototipo a escala real.
Segunda entrega	Jueves 14 de mayo de 2026	Implementación inicial del sistema, lectura de sensores, control de actuadores y definición de la plataforma de visualización IoT: Firebase o MQTT.
Entrega final	Miércoles 27 de mayo de 2026	Prototipo funcional integrado, monitoreo remoto, bitácora completa, repositorio GitHub actualizado, presentación final y demostración del sistema.

XIX. ESTRUCTURA SUGERIDA DEL REPOSITORIO

```

proyecto-final-sistemas-embebidos/
|
|-- README.md
|-- docs/
|   |-- requisitos.md
|   |-- arquitectura.md
|   |-- bitacora.md
|   |-- evidencias/
|
|-- firmware/
|   |-- src/
|   |-- include/
|   |-- platformio.ini
|   |-- README.md
|
|-- hardware/
|   |-- diagrama_conexiones.png
|   |-- lista_componentes.md
|
|-- iot/
|   |-- firebase_config_example.json
|   |-- mqtt_topics.md
|   |-- dashboard_evidencias.md
|
|-- presentacion/
|   |-- slides.pdf

```

XX. RÚBRICA DE EVALUACIÓN

Cuadro IV
RÚBRICA PROPUESTA PARA EL PROYECTO FINAL

Criterio	Peso
Levantamiento de requisitos y análisis del problema	15 %
Diseño de arquitectura hardware/software	15 %
Implementación de lectura de sensores	15 %
Control de actuadores y lógica embebida	15 %
Implementación IoT: Firebase o MQTT	15 %
Buenas prácticas de código y uso de GitHub	10 %
Bitácora, evidencias y gestión ágil del proyecto	10 %
Presentación y demostración final	5 %
Total	100 %

XXI. PRODUCTO ESPERADO

Al finalizar el proyecto, cada grupo deberá demostrar un prototipo funcional capaz de:

- Medir variables ambientales y del sustrato.
- Enviar datos a una plataforma IoT.
- Activar automáticamente el riego por goteo.
- Activar un sistema de control térmico.
- Mostrar el estado del sistema de forma remota.
- Evidenciar el desarrollo mediante GitHub y bitácora.
- Explicar técnicamente las decisiones de diseño tomadas.

XXII. CONCLUSIONES

El proyecto final propuesto permite integrar los conceptos fundamentales del curso de Sistemas Embebidos en una aplicación práctica orientada al Internet de las Cosas. La solución exige que los estudiantes realicen un proceso completo de ingeniería, desde el levantamiento de requisitos hasta la validación de un prototipo funcional.

Además, el proyecto fortalece competencias asociadas al diseño de sistemas mecatrónicos, el uso de documentación técnica, la programación estructurada, la integración hardware/software, la comunicación IoT y la apropiación de tecnologías aplicadas a contextos de sostenibilidad y agricultura inteligente.

REFERENCIAS

- [1] P. Marwedel, *Embedded System Design: Embedded Systems Foundations of Cyber-Physical Systems, and the Internet of Things*, 3rd ed. Cham, Switzerland: Springer, 2018.
- [2] S. Salas Arriarán, *Todo sobre sistemas embebidos: Arquitectura, programación y diseño de aplicaciones prácticas con el PIC18F*. Lima, Perú: Universidad Peruana de Ciencias Aplicadas, 2015.
- [3] A. S. Berger, *Embedded Systems Design: An Introduction to Processes, Tools, and Techniques*. Lawrence, KS, USA: CMP Books, 2002.
- [4] E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd ed. Cambridge, MA, USA: MIT Press, 2017.
- [5] J. W. Valvano, *Embedded Microcomputer Systems: Real Time Interfacing*, 3rd ed. Stamford, CT, USA: Cengage Learning, 2011.
- [6] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY, USA: Springer, 2011.
- [7] OASIS, "MQTT Version 5.0," OASIS Standard, Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

- [8] HiveMQ, “MQTT Essentials: A Lightweight IoT Protocol,” 2020. [Online]. Available: <https://www.hivemq.com/mqtt-essentials/>
- [9] Google, “Firebase Realtime Database Documentation,” Google Firebase. [Online]. Available: <https://firebase.google.com/docs/database>
- [10] Espressif Systems, “ESP32 Series Datasheet,” Espressif Systems. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [11] Raspberry Pi Ltd., “RP2040 Datasheet: A microcontroller by Raspberry Pi,” Raspberry Pi. [Online]. Available: <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>
- [12] STMicroelectronics, “STM32 Nucleo Boards,” STMicroelectronics. [Online]. Available: <https://www.st.com/en/evaluation-tools/stm32-nucleo-boards.html>
- [13] J. M. Talavera *et al.*, “Review of IoT applications in agro-industrial and environmental fields,” *Computers and Electronics in Agriculture*, vol. 142, pp. 283–297, Nov. 2017, doi: 10.1016/j.compag.2017.09.015.
- [14] S. Madakam, R. Ramaswamy, and S. Tripathi, “Internet of Things (IoT): A literature review,” *Journal of Computer and Communications*, vol. 3, no. 5, pp. 164–173, 2015, doi: 10.4236/jcc.2015.35021.
- [15] L. M. Dang, M. J. Piran, D. Han, K. Min, and H. Moon, “A survey on Internet of Things and cloud computing for healthcare,” *Electronics*, vol. 8, no. 7, Art. no. 768, 2019, doi: 10.3390/electronics8070768.
- [16] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum*. 2020. [Online]. Available: <https://scrumguides.org/>